Spark's Basic Architecture

Spark architecture explains how data is processed in Spark.

In simple words, Spark works like a team system. One main controller gives instructions, and multiple workers do the actual work.

The main parts are:

1. Driver (main brain)
2. Workers (do the work)
3. Cluster Manager (manages resources)

Example:

Imagine a teacher giving homework:

1. Teacher = Driver
2. Students = Workers
3. Homework = Data

Teacher divides work → students complete → results collected.

Working Principle

- Data is divided into smaller parts
- Tasks are executed in parallel

Key Features

- High speed processing
- Scalable (can add more machines

Spark's Basic Architecture

Cluster Manager:

Cluster Manager is responsible for managing resources in a Spark cluster.

It controls how tasks are assigned to different machines and ensures efficient usage of CPU and memory.

It acts as a central authority that decides which worker node should perform a particular task.

It also monitors the system and balances workload among nodes.

There are different types of cluster managers such as Standalone, YARN, and Mesos.

Example

In a company:

- Manager assigns work to employees
- Ensures work is evenly distributed

Working Principle

- Allocates CPU and memory to tasks
- Assigns tasks to worker nodes

Key Features

- Prevents system overload
- Improves performance

Spark's Basic Architecture

Spark Session:

SparkSession is the main entry point to work with Apache Spark. It is the first thing we create when starting a PySpark program.

Without SparkSession, we cannot perform any operation such as reading data, creating DataFrames, or running queries.

In earlier versions of Spark, different contexts like SQLContext and HiveContext were used separately. But now, SparkSession combines all these functionalities into one single interface.

This makes it easier for developers to work with Spark using a unified approach.

Example

It is similar to switching ON a system before starting work. Just like we cannot use a computer without turning it on, we cannot use Spark without creating a SparkSession.

- **Working Principle**
  - Initializes Spark environment
  - Provides access to all Spark functionalities

- **Key Features**
  - Easy to use
  - Supports multiple operations

Spark's Basic Architecture

Spark Context:

SparkContext is one of the core components of Apache Spark. It acts as a connection between the user application and the Spark cluster.

It is responsible for distributing data and tasks across different worker nodes.

It takes the instructions from the driver program and sends them to the worker nodes for processing.

Example

Imagine you are placing an order in a restaurant:

- You (user) give the order
- Waiter takes the order to kitchen
- Kitchen prepares food

Working Principle

- Establishes connection with Spark cluster
- Distributes tasks to worker nodes
- Collects results after execution

Key Features

- Controls data distribution
- Handles communication between nodes
- Supports parallel processing

Spark's Basic Architecture

Driver Node:

Driver Node is the main control unit of a Spark application. It is responsible for running the user program and managing the entire execution process.

It converts the user's code into tasks and sends them to worker nodes.

The driver node keeps track of all operations and collects results after processing. It also maintains information about the progress of tasks and handles errors if any occur.

Example

Think of a classroom:

- Teacher gives instructions
- Students perform tasks
- Teacher checks results

Working Principle

- Breaks the program into smaller tasks
- Sends tasks to worker nodes
- Monitors execution

Key Features

- Acts as the brain of the system
- Controls execution flow
- Collects final output

Spark's Basic Architecture

Worker Node:

Worker Nodes are the systems where actual data processing takes place.

They receive tasks from the driver node and execute them.

Each worker node contains executors that perform computations.

Worker nodes process data in parallel, which increases speed and efficiency.

They also store intermediate results during execution.

Worker Nodes are responsible for actual computation and play a key role in distributed data processing.

Example

Students solving problems given by teacher.

Working Principle

- Receives tasks from driver
- Processes data
- Sends results back

Key Features

- Executes tasks in parallel
- Handles large datasets
- Stores temporary data

Spark's Basic Architecture

Client Mode:

Client Mode is one of the ways to run a Spark application. In this mode, the driver program runs on the user's local machine, while the worker nodes run in the cluster.

This mode is mainly used for development and testing because it allows the user to directly control the execution and easily debug errors.

Good for learning and testing

Client Mode is useful for beginners and development purposes as it provides better control and visibility.

Example

Teacher sitting in the classroom and controlling students directly.

Working Principle

- Driver runs locally
- Tasks executed on cluster
- Results sent back to local system

Key Features

- Easy debugging
- Direct control over execution
- Suitable for small tasks

Spark's Basic Architecture

Cluster Mode:

Cluster Mode is another way to run Spark applications. In this mode, the driver program runs inside the cluster instead of the local machine.

This allows the application to run independently without depending on the user's system.

It is mainly used for large-scale production systems where performance and reliability are important.

Cluster Mode is ideal for real-world applications where large data needs to be processed efficiently and reliably.

Example

Teacher working from office while students work in classrooms independently.

Working Principle

- Driver runs inside cluster
- Tasks executed by worker nodes
- Results managed within cluster

Key Features

- Fully distributed execution
- Better resource utilization
- High reliability